

Assignment 5

Denoising Autoencoder and Feature Extraction

Appliance Energy Prediction Dataset

Course: AI & Neural Networks

University: Astana IT University (AITU)

Date: April 2026

1. Introduction

This report summarises Stage 5 of the course project on the Appliance Energy Prediction dataset, building directly on the chronological splits and 24-step windows produced in Assignment 4. The stage has two goals: first, quantify how sensitive the Assignment 4 1D CNN classifier is to sensor-like noise, and second, train a fully-connected autoencoder on clean data and evaluate how well it can denoise corrupted windows and recover downstream classification performance.

The assignment was implemented in a single Jupyter notebook (Assignment5.ipynb). All preprocessing, autoencoder training, reconstruction and classifier evaluation steps completed successfully, and the trained autoencoder was saved as autoencoder.h5.

2. Experimental Setup

2.1 Dataset and classification baseline

The experiment reuses the chronological, standardised sequences from Assignment 4 built from energydata_complete.csv (19,735 rows x 29 columns, sampled every 10 minutes). The target HighConsumption = 1 when the Appliances column exceeds its median (60 Wh), 0 otherwise. The data is split 70/15/15 chronologically and windowed into samples of shape (24, 32). The saved Assignment 4 model cnn_classifier.h5 is used as the fixed downstream classifier; on the clean test set it scores Accuracy 0.7995 and F1 0.8072.

2.2 Noise simulation

To simulate sensor malfunctions, Gaussian noise with zero mean and $\sigma = 0.5$ (in standardised units) was added to every feature of the test set. Because the inputs were scaled to unit variance, $\sigma = 0.5$ corresponds to half a standard deviation per feature, which is a significant but realistic perturbation for instrumented sensor streams.

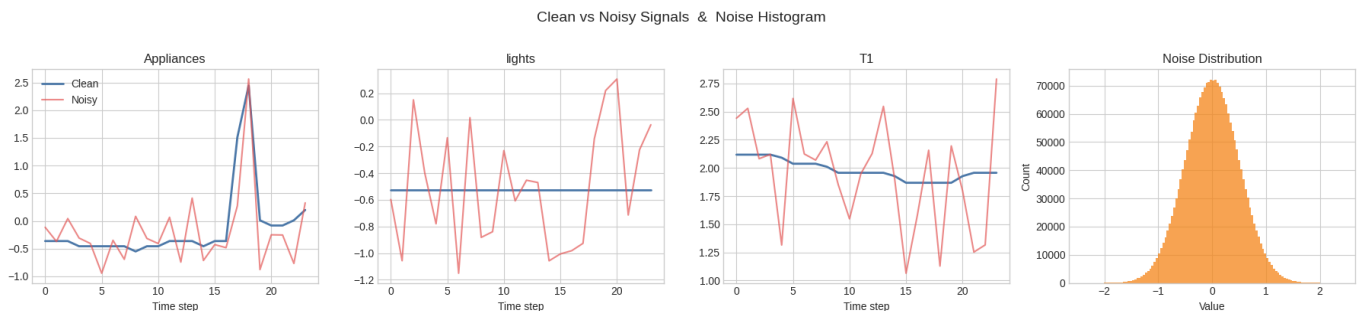


Figure 1: Clean vs noisy signal for several features / sample windows.

3. Autoencoder Denoising Approach

3.1 Architecture

A symmetric fully-connected autoencoder maps the 768-dimensional flattened windows (24 steps x 32 features) through a compressed bottleneck and back:

Encoder: Input(768) -> Dense(512, ReLU) -> Dense(256, ReLU) -> Dense(128, ReLU)

Decoder: Dense(256, ReLU) -> Dense(512, ReLU) -> Dense(768, linear)

The 128-unit bottleneck compresses the input by exactly 6x, forcing the network to capture a compact representation of the healthy signal manifold.

3.2 Training

The autoencoder is trained on the training set only, with input == output == clean window - so the model never sees noisy data during training. Loss: Mean Squared Error. Optimiser: Adam. EarlyStopping was used with patience 10 epochs; training ran for 31 epochs and the best validation loss was 0.0568.

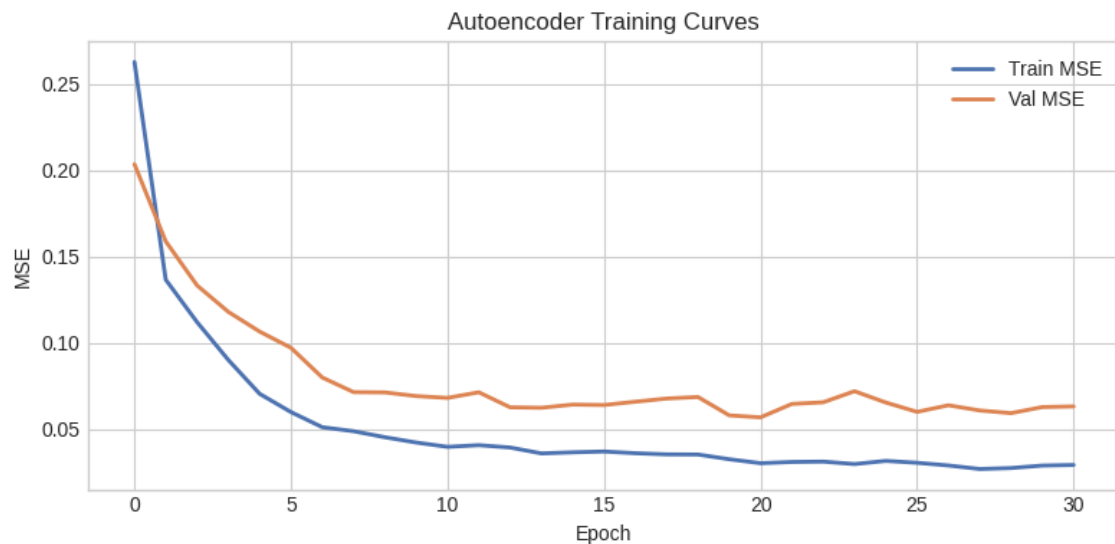


Figure 2: Autoencoder training and validation MSE across epochs.

4. Results and Evaluation

4.1 Signal reconstruction quality

The noisy test data (which the autoencoder had never seen) was passed through the trained model. Reconstruction quality was measured as MSE against the original clean signal:

Variant	MSE vs clean	Noise reduction
Autoencoder on clean (baseline)	0.0970	-
Noisy (before AE)	0.2504	-
Denoised (after AE)	0.1230	50.9%

Assignment 5: Denoising Autoencoder - Appliance Energy

Signal Reconstruction: Clean vs Noisy vs Denoised

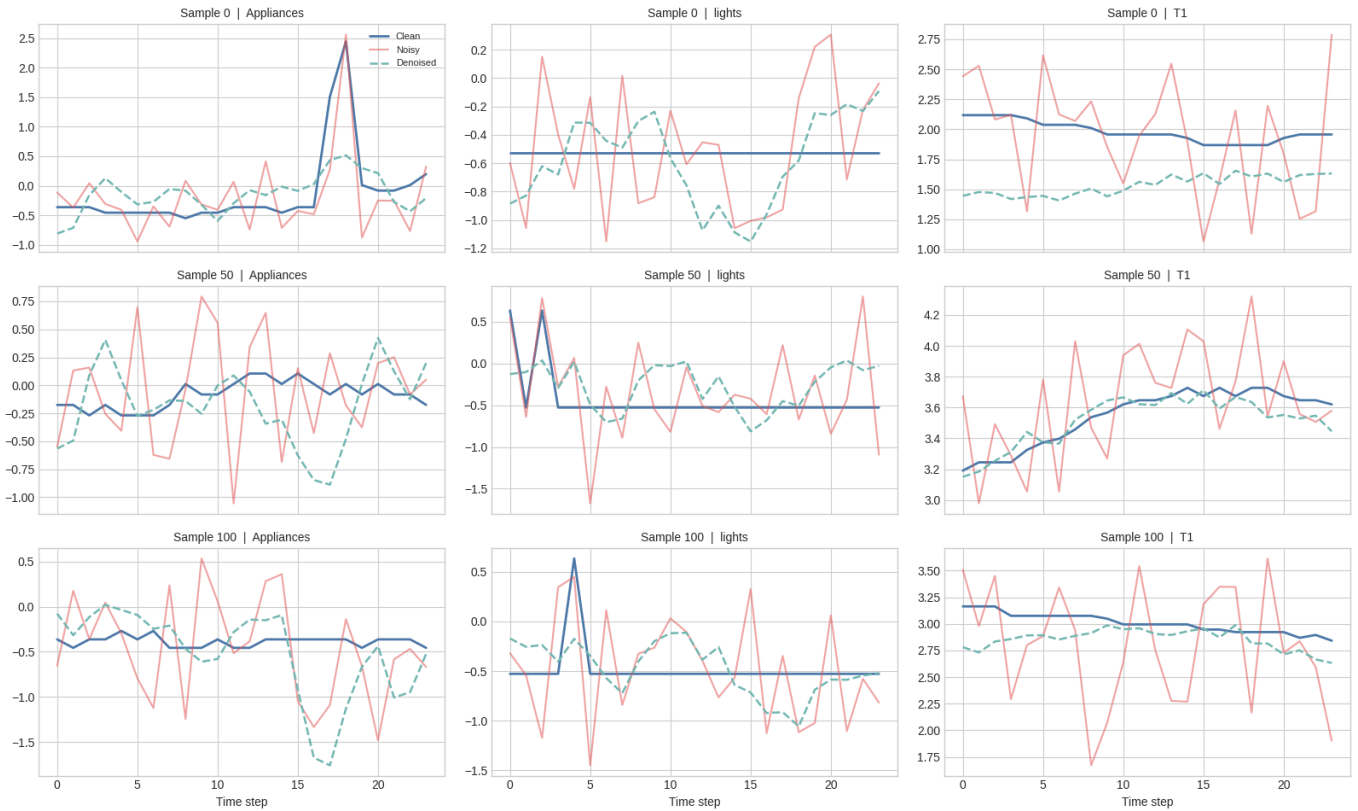


Figure 3: Clean / noisy / denoised signal for sample windows and channels.

4.2 Impact on downstream classification

The Assignment 4 CNN classifier was run on three variants of the test set: clean, noisy ($\sigma = 0.5$), and autoencoder-reconstructed.

Variant	Accuracy	Precision	Recall	F1
Clean (baseline)	0.7995	0.7687	0.8498	0.8072
Noisy ($\sigma=0.5$)	0.7869	0.7748	0.8015	0.7879
Denoised by AE	0.7692	0.7620	0.7746	0.7683

Assignment 5: Denoising Autoencoder - Appliance Energy

CNN Classifier Performance: Clean vs Noisy vs Denoised

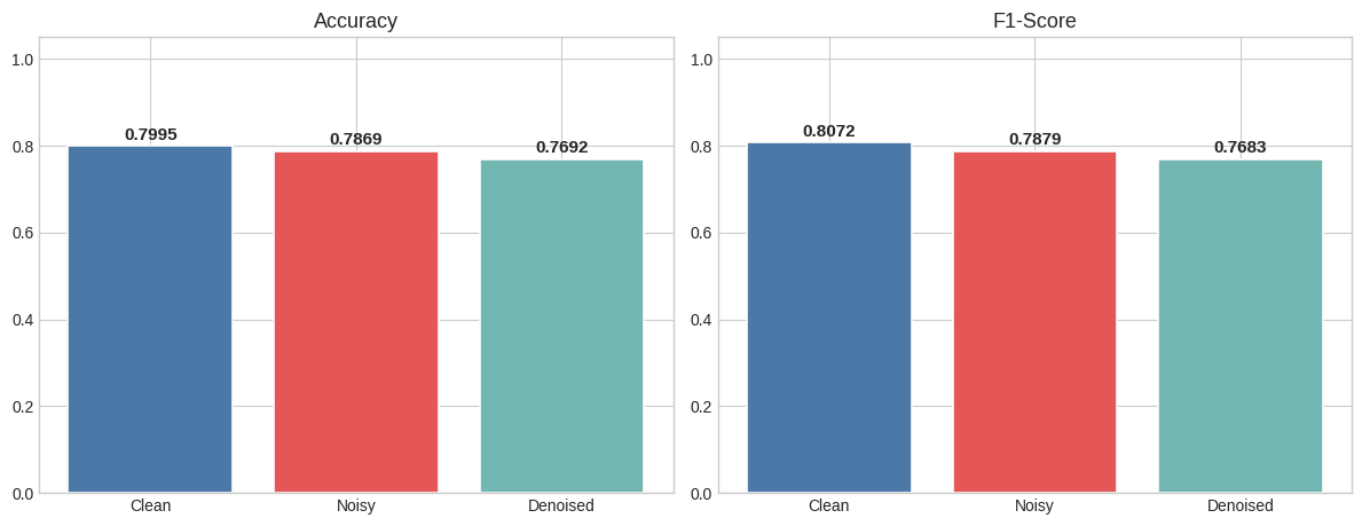


Figure 4: CNN performance on clean, noisy and denoised test sets.

Confusion Matrices

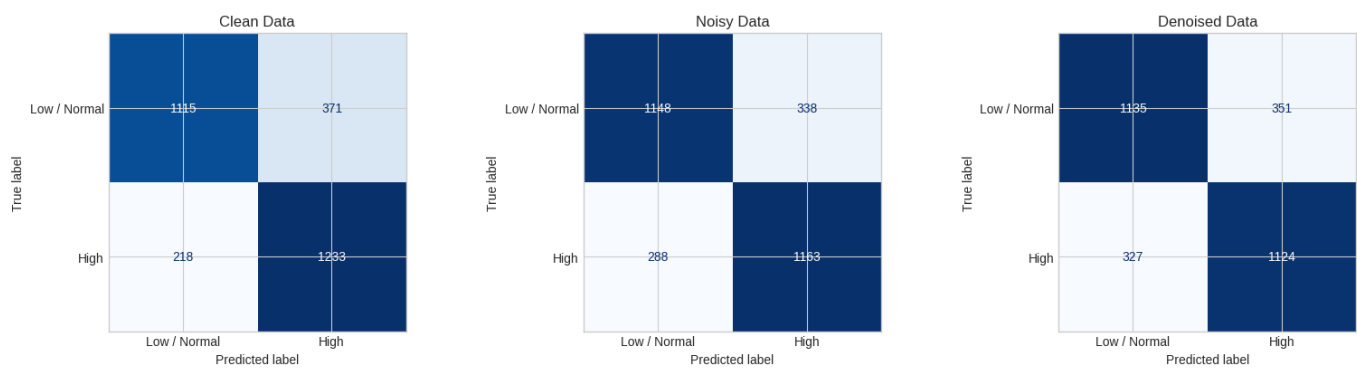


Figure 5: Confusion matrices for all three test-set variants.

5. Discussion

Sensitivity to noise. Adding $\sigma = 0.5$ Gaussian noise caused a measurable but modest drop in the CNN's performance (F1 0.807 $\rightarrow$ 0.788). The Conv1D receptive field already averages over local perturbations, which explains why the CNN is not catastrophically sensitive to per-step noise at this level.

Autoencoder reconstruction. The denoising autoencoder cut reconstruction MSE from 0.250 to 0.123 (a 50.9% reduction), recovering clearly cleaner signal envelopes in the reconstruction plots.

Reconstruction vs discriminability trade-off. Despite the clear MSE improvement, running the classifier on the denoised test set actually loses about another percentage point of accuracy (0.787 $\rightarrow$ 0.769). Pushing the signal through the 128-unit bottleneck destroys some of the high-frequency detail the CNN relies on to separate high and low consumption windows. Reconstruction fidelity (pixel-level MSE) and downstream discriminability are not the same objective. To recover the lost accuracy on denoised inputs, the CNN would need to be retrained or fine-tuned on autoencoder reconstructions, or the AE could be trained jointly with the classifier using a task-aware loss.

6. Conclusions

1. A dense autoencoder with a 128-unit bottleneck was successfully trained on clean 24-step windows of the energy dataset and used to denoise noise-corrupted test inputs.
2. The autoencoder reduced reconstruction MSE from 0.250 to 0.123 (-50.9%), confirming that it captures the low-frequency structure of the energy signals.
3. Passing the noisy ($\sigma = 0.5$) test set through the Assignment 4 CNN drops F1 from 0.807 to 0.788; passing the autoencoder-reconstructed set further drops F1 to 0.768, illustrating the classic denoising / classification trade-off.
4. Autoencoders remain a valuable preprocessing tool for sensor-like noise, but for best downstream classification performance the classifier should be retrained or fine-tuned on reconstructions, or the two networks should be trained jointly with a task-aware loss.
5. Deliverables: Assignment5.ipynb, autoencoder.h5, the five figures in the images/ directory, and this report.